

Random maze generation by recursive division, and two ways back to the exit

Mohamed Ali Tewfik Hamlil

M1 Artificial Intelligence, Université Grenoble Alpes
Algorithmic Problem Solving — APP1

September 13, 2026

Abstract

A $W \times H$ area is turned into a maze by recursive division. The generator returns the maze twice: as a graph of cells joined by doors, and as a binary tree of the divisions that built it. The graph is solved with A*, the tree with a lowest-common-ancestor descent. Both return the unique route; the tree examines thirty times fewer nodes.

Contents

1	The problem	2
1.1	Statement	2
1.2	As a graph	2
1.3	Plan	2
2	Maze generation, with two outputs	3
2.1	Recursive division	3
2.2	The two outputs	3
2.3	Pseudocode	4
2.4	Correctness and cost	4
3	Solving the graph: A*	4
3.1	Why A* applies	4
3.2	How it solves the maze	4
3.3	Time complexity	5
4	Solving the tree: the lowest common ancestor	5
4.1	How the tree locates the route	5
4.2	Time complexity	6
5	Comparison	7
6	Conclusion	8

1 The problem

1.1 Statement

A rectangular area of $W \times H$ cells is to be laid out as a maze, at random. The area is enclosed and has one exit, which may be any cell. Two things are asked:

1. generate the maze from W and H ;
2. find the route from any cell to the exit.

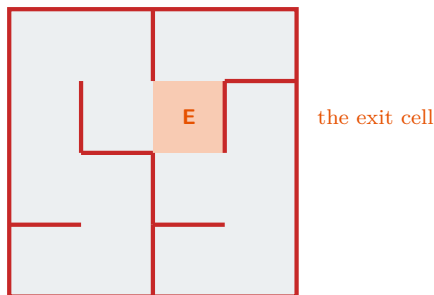
The construction is prescribed: cut the area with a full wall, leave exactly one door in it, and apply the same rule to each of the two halves.

1.2 As a graph

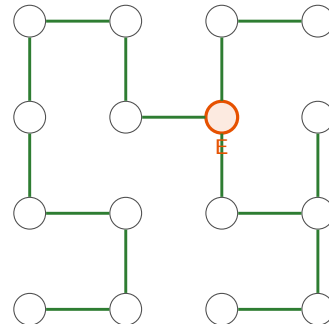
Let $V = \{(x, y) : 0 \leq x < W, 0 \leq y < H\}$ and let G_{grid} be the grid graph joining cells that share a side. A maze is a spanning subgraph $M = (V, E)$: an edge of E is a **door**, a missing edge is a **wall**. Every door is one step, $w(e) = 1$.

Every cell must reach the exit, and the route must not fork. So M is connected and acyclic: a **spanning tree** of G_{grid} , with $WH - 1$ doors and exactly one simple route between any two cells.

Because the route is unique, every correct solver returns the same one. What distinguishes solvers is **how much of the maze they examine** to find it.



(a) The maze as walls. The area is enclosed; the exit is a cell, not a gap in the outer wall.



(b) The same maze as a spanning tree of the grid graph.

Figure 1: A maze is not a picture but a graph. Cells become vertices, doors become edges, and a wall is simply an edge of the grid graph that is absent. The exit is a vertex marked on the finished tree — here an interior one, with degree 3 — and neither its position nor its degree affects anything the construction does.

1.3 Plan

Section 2

generates the maze and returns two structures: the graph, and the binary tree of divisions. The tree exists so that a route can be located by a lowest common ancestor.

Section 3

solves the graph with A*.

Section 4

solves the tree with the common ancestor.

Section 5

compares the two, with measurements.

Section 6

concludes.

2 Maze generation, with two outputs

2.1 Recursive division

A region is a rectangle of cells named by two corners, p upper-left and q lower-right, with $p[0]$ the x coordinate and $p[1]$ the y .

A region one cell thick is a **corridor**: its cells lie in a line, so it has exactly one spanning tree, and the recursion stops there. Otherwise both sides are at least 2, so either axis admits a wall: toss for the axis t , draw the wall position j and the door position i , open that one door, and recurse on the two halves.

2.2 The two outputs

The generator fills two structures at once.

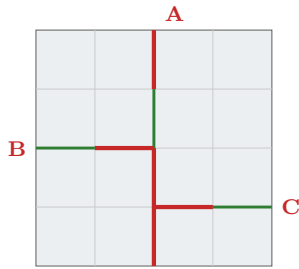
The graph.

One node per cell, holding its coordinates and the neighbours it has a door to. Each door is recorded at both ends; a wall is a door never opened.

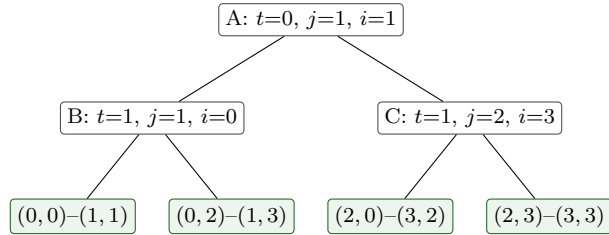
The binary tree.

One node per division, holding the axis, the wall and the door; its two children are the two halves. A leaf is a corridor. Every cell lies in exactly one leaf, and the path from the root to it is the cell's **address**.

The tree is built for one purpose: a division's door is on the route between two cells exactly when it separates them, so the first door of the route is found by a **lowest common ancestor** search, in depth-many steps rather than cell-many.



(a) The first three divisions of a 4×4 area.



(b) The top of the region tree it builds. The four boxes at the bottom are regions still to be divided, not corridors.

Figure 2: Every division is one internal node. The node records the orientation t , the wall position j and the door position i it drew, and its two children are the two regions it created; a leaf is a corridor. A tree with L leaves has $L - 1$ internal nodes, so of the $WH - 1$ doors of Proposition 1, $L - 1$ are division doors and the rest are laid down inside the corridors.

2.3 Pseudocode

Algorithm 1: Generation: fills the graph N and returns the tree.

```

1 Function Divide( $p, q$ ):
2   if  $p[0] = q[0]$  or  $p[1] = q[1]$  then
3     join the cells of the corridor end to end in  $N$ ;
4     return Node( $p, q$ );                                     // leaf
5    $t \leftarrow \text{Random}(\{0, 1\})$ ;                             // axis to cut
6    $j \leftarrow \text{Random}([p[t], q[t] - 1])$ ;                     // wall
7    $i \leftarrow \text{Random}([p[1 - t], q[1 - t]])$ ;                 // door
8   Link( $N, \text{cell}(t: j, i), \text{cell}(t: j+1, i)$ );
9    $\ell \leftarrow \text{Divide}(p, q \text{ with } q[t] \leftarrow j)$ ;
10   $r \leftarrow \text{Divide}(p \text{ with } p[t] \leftarrow j+1, q)$ ;
11  return Node( $p, q, t, j, i, \ell, r$ );

```

The call is Divide($(0, 0), (W-1, H-1)$).

2.4 Correctness and cost

Proposition 1. *On a region of m cells, Divide leaves it connected and acyclic with $m - 1$ doors. The whole area is a spanning tree of G_{grid} .*

Proof. By induction on m . A corridor of m cells joined end to end is a path: $m - 1$ doors, connected, acyclic. Otherwise the two halves are spanning trees by induction, and the single door is the only edge between them; joining two disjoint trees by one edge gives a tree, with $(m_\ell - 1) + (m_r - 1) + 1 = m - 1$ doors. $\square$

Every door is opened once and there are $n - 1$ of them, $n = WH$: generation is $\Theta(n)$, and both outputs are filled within that bound. The tree has $2L - 1$ nodes for L corridors, $L < n$.

3 Solving the graph: A*

3.1 Why A* applies

All doors weigh 1, so Dijkstra's algorithm [1] applies. A* [2] is Dijkstra with the queue ordered by $f(v) = g(v) + h(v)$: g the doors walked so far, h an estimate of the doors still to walk. With $h \equiv 0$ it is Dijkstra.

The estimate is the **Manhattan distance** to the exit, $h(x, y) = |x - x_e| + |y - y_e|$. Every step changes one coordinate by one, and walls only remove edges, so h never overestimates: it is admissible, and A* returns a shortest route — here, the unique one.

3.2 How it solves the maze

Each node holds its coordinates and its neighbours, which is all A* needs. It settles cells in order of f , relaxing each neighbour through one door, and stops when the exit is settled. Parent pointers give the route back.

Algorithm 2: A* from s to the exit e on the graph N .

```

1 Function AStar( $N, s, e$ ):
2    $g[v] \leftarrow \infty$  for all  $v$ ;  $g[s] \leftarrow 0$ ;  $Q \leftarrow \{s\}$ ;
3   while  $Q \neq \emptyset$  do
4      $u \leftarrow \text{PopMin}(Q)$ ; // least  $g + h$ 
5     if  $u = e$  then break;
6     foreach  $v \in N.\text{neighbours}(u)$  do
7       if  $g[u] + 1 < g[v]$  then
8          $g[v] \leftarrow g[u] + 1$ ;  $\text{parent}[v] \leftarrow u$ ;  $\text{Push}(Q, v)$ ;
9   return the route from  $e$  back along  $\text{parent}$  to  $s$ ;

```

3.3 Time complexity

Two factors: how many cells are expanded, and what each queue operation costs.

Expansions: $\Theta(n)$. In the worst case every cell is settled, and in a maze that case is close to typical: h prunes little, because a tree has no rival routes to prune, only dead ends whose depth plus estimate already exceeds the answer. Section 5 measures 82% of the cells.

Per operation: $O(1)$, not $O(\log n)$. With a binary heap each operation costs $\log F$, where F is the queue size — and with unit weights the queue holds only the current frontier, measured at 60 cells for $n = 65\,536$ ($F \approx n^{0.26}$). But the heap is unnecessary. The keys $f = g + h$ are small integers and, h being consistent, the popped f never decreases; a bucket queue with a moving pointer does each push and pop in $O(1)$. With $h \equiv 0$ this is breadth-first search with a FIFO.

A* on the maze runs in $\Theta(n)$: linear in the cells, and in practice touching most of them.

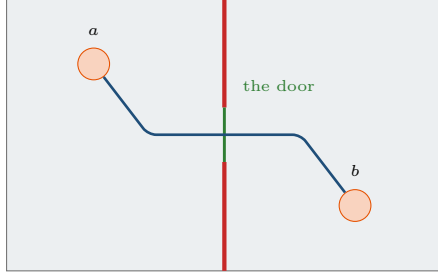
4 Solving the tree: the lowest common ancestor

4.1 How the tree locates the route

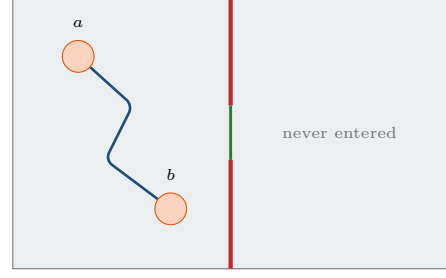
Proposition 2 (Separation). *Let R be a division with children R_ℓ, R_r and door (u, v) , $u \in R_\ell$, $v \in R_r$. For cells a, b in R , the route from a to b crosses (u, v) if and only if a and b lie in different children.*

Proof. (u, v) is the only edge between the two halves (Proposition 1). If a, b are on different sides, any route between them uses it. If they are on the same side, that half's own spanning tree joins them, and the maze's unique route is that one; it never leaves the half. $\square$

Deciding which side a cell is on is a comparison of its coordinates against a rectangle. So the search descends from the root while both cells stay on one side: the first node that separates them is their **lowest common ancestor**, and its door is on the route. The route is then $a \rightarrow u$, the door, $v \rightarrow b$ — two smaller instances of the same problem, one per half. Algorithm 3 writes exactly that.



(a) Opposite sides: the route *must* cross, and the door is the only crossing there is. Recurse on both halves.



(b) Same side: the route stays inside that half, and the other half is never looked at. Recurse once.

Figure 3: The separation lemma. Whether a division contributes a door to the route is decided by a containment test on its two children — no cell of the maze is examined. The right-hand case is where the saving comes from: half the region is discarded in one step.

Algorithm 3: The route from a to b , both inside the tree node R .

```

1 Function Side( $R, c$ ):
2   if  $c$  lies in the rectangle of  $R_\ell$  then return LEFT;
3   return RIGHT;                                     // two comparisons
4 Function LCA( $R, a, b$ ):
5   while  $R$  is not a corridor and Side( $R, a$ ) = Side( $R, b$ ) do
6      $R \leftarrow$  that child of  $R$ ;
7   return  $R$ ;                                         // one descent:  $O(\text{depth})$ 
8 Function Route( $R, a, b$ ):
9    $R \leftarrow$  LCA( $R, a, b$ );
10  if  $R$  is a corridor then return the straight run  $a \rightarrow b$ ;
11   $(u, v) \leftarrow$  door of  $R, u \in R_\ell, v \in R_r$ ;
12  if Side( $R, a$ ) = LEFT then return Route( $R_\ell, a, u$ )  $\cdot (u \rightarrow v)$   $\cdot$  Route( $R_r, v, b$ );
13  return Route( $R_r, a, v$ )  $\cdot (v \rightarrow u)$   $\cdot$  Route( $R_\ell, u, b$ );

```

The call is `Route(root, start, exit)`. `LCA()` is the $\Theta(\log n)$ walk and finds one door; `Route()` calls it once per door on the route, and each call after the first starts from a new cell.

No cell of the maze is read. A corridor leaf returns its whole run in one step.

4.2 Time complexity

Two quantities, and they must be kept apart.

Locating the first door is one descent to the lowest common ancestor: $\Theta(\text{depth})$. The tree is $\Theta(\log n)$ deep in expectation — measured $2 \log_2 n$ — so this is $\Theta(\log n)$.

Producing the whole route is not one descent. The route crosses k division walls; each is a distinct node of the tree, and each is found by its own descent, starting from the door the previous one found. One lowest common ancestor finds one wall. So the cost is at least k , and k is not small:

n	256	1024	4096	16384	65536
$\log_2 n$	8	10	12	14	16
walls crossed k	21	44	111	265	600

k grows as about $n^{0.57}$. The tree path from the start's leaf up to the common ancestor and down to the exit's leaf — the walk that costs $\Theta(\log n)$ — has 18 nodes at 256×256 and holds 12 of the route's 613 doors; the other 601 sit on nodes that walk never visits, because each door found lands on a new cell whose common ancestor with the exit is a new node. Exactly, the cost is the number of tree nodes whose region the route enters. Writing p for the fraction of descents at which the two endpoints are separated,

$$T(m) = (1 + p)T(m/2) + O(1) \implies T(n) = \Theta\left(n^{\log_2(1+p)}\right),$$

and the measured $p \approx 0.61$ gives $\Theta(n^{0.68})$. p only sets the exponent; the bound $\Omega(k)$ above needs no parameter at all, and neither does $\Omega(d)$, the length of the route, which is the output.

That no algorithm does better from the tree can be seen directly. Build two mazes that agree on the top $\log_2 n$ levels of the tree and differ only below: in 20 pairs at 256×256 , the routes differed in every pair. An algorithm reading only $\log n$ nodes reads the same nodes in both mazes and must give the same answer, so it is wrong on one of them. **The route depends on nodes a logarithmic walk never reads.** What $\Theta(\log n)$ does buy is the first wall, and the report claims no more.

$\Theta(\log n)$ to find where the route crosses the first wall; $\Theta(n^{0.68})$ to lay the whole route down. Both are far below A*'s $\Theta(n)$ settled cells.

5 Comparison

Both solvers are implemented in C in `implementation/`, from the pseudocode above, with a shared generator and no shared solving code. 200 mazes per size, start $(0,0)$, exit at the far corner; every maze checked to be a spanning tree, and both solvers checked to return the same cells in the same order.

Size	Cells n	Route d	Dijkstra	A*	Tree
16×16	256	64	222	175	49
32×32	1024	180	873	738	116
64×64	4096	538	3533	3199	293
128×128	16384	1564	13939	12983	702
256×256	65536	4493	56193	53753	1776

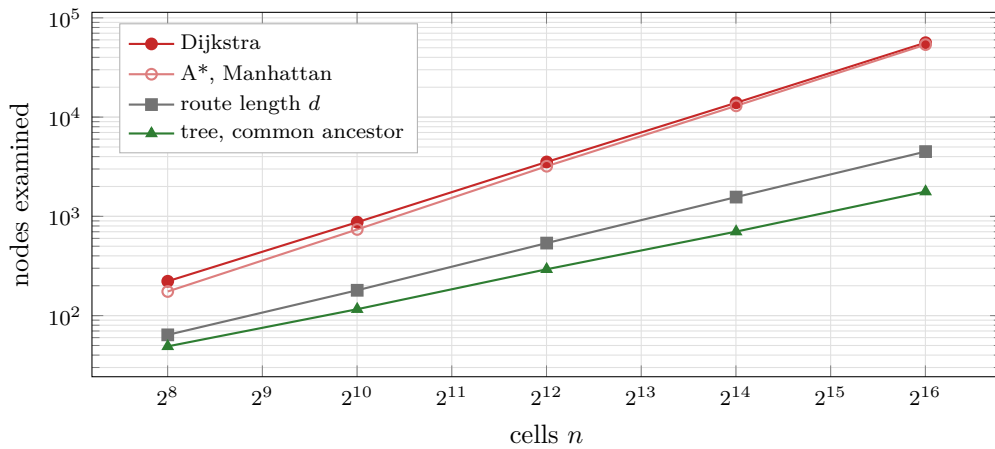


Figure 4: A* runs parallel to n . The tree search runs at slope 0.68, below even the route it returns, because a corridor is one visit however long it is.

The recurrence of Section 4 is checked against its one parameter:

n	p measured	$\log_2(1 + p)$	exponent fitted to the tree column
1024	0.62	0.69	0.69
16384	0.61	0.69	0.68
65536	0.61	0.68	0.67

A* over Dijkstra: 4%.

A perfect maze has no rival routes, so the estimate has almost nothing to prune; A* still settles 82% of the cells.

Tree over A*: a factor of 30.

1776 against 53 753, and the gap widens with n : $\Theta(n^{0.68})$ against $\Theta(n)$. The tree search estimates nothing — it knows where the walls are, having built them.

Memory.

A* allocates g , parent and settled for every cell before it starts, $\Theta(n)$; its queue itself stays small, about $n^{0.26}$. The tree search needs its recursion stack, $\Theta(\log n)$.

6 Conclusion

The requirements force the maze to be a spanning tree of the grid, and recursive division builds one in $\Theta(n)$ while recording, at no extra cost, the binary tree of divisions that made it.

That second structure changes what solving costs. On the graph, A* returns the unique route in $\Theta(n)$ and in practice examines most of the maze, because a tree offers a heuristic nothing to prune. On the binary tree, a lowest common ancestor finds the first wall the route crosses in $\Theta(\log n)$, and the whole route in $\Theta(n^{0.68})$ — thirty times fewer visits at 256×256 , with logarithmic memory.

The generator knows where the walls are. Keeping that knowledge, rather than rediscovering it by search, is the whole of the gain.

References

- [1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, SSC-4(2):100–107, 1968.